

Java Backend Architecture

Interview Series

Chapter 8.9

Password Hashing & Cryptography

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 8.9 — Password Hashing & Cryptography

Introduction

Cryptography is the foundation of application security. Understanding the difference between fast hashes (MD5/SHA-256 — for checksums) and adaptive password hashes (BCrypt/Argon2id — for passwords) is critical. For data encryption, AES-256-GCM provides authenticated symmetric encryption, while RSA-OAEP and ECDSA provide asymmetric encryption and digital signatures. Key management (AWS KMS, HashiCorp Vault) is as important as the algorithms.

Password Hashing Algorithms

MD5/SHA-1
BROKEN
Never for passwords
GPU: 10B/sec

SHA-256
FAST HASH
Not for passwords
Needs salt manually

BCrypt
RECOMMENDED
Cost factor 10-12
60-char output

Argon2id
BEST (2023)
PHC winner
memory+time+parallelism

SCrypt
MEMORY-HARD
N=2^17, r=8, p=1
Blocks GPU attacks

PBKDF2
FIPS COMPLIANT
PBKDF2WithHmacSHA512
600k iterations (NIST)

Adaptive hashing: increase cost factor as hardware improves. Target: >100ms hash time on the server.
Salting: BCrypt/SCrypt/Argon2 auto-generate and store unique salt. PBKDF2: generate random 16-byte salt separately.

Algorithm	Type	Parameters	Spring Class	Recommendation
BCrypt	Adaptive, salted	cost 10-12 (2^cost iterations)	BCryptPasswordEncoder	Good default choice
Argon2id	Memory+time+parallelism	memory=19MiB, iter=2, par=1	Argon2PasswordEncoder	Best for new systems
SCrypt	Memory-hard	N=2^17, r=8, p=1	SCryptPasswordEncoder	Good, complex params
PBKDF2	Iterated HMAC	SHA-512, 600k iterations	Pbkdf2PasswordEncoder	FIPS/NIST compliant
MD5/SHA-1	Fast hash	None	N/A	NEVER for passwords
SHA-256	Fast hash	None	N/A	NEVER alone (needs salt + iterations)

```
// Spring Security password encoding @Bean public PasswordEncoder passwordEncoder() { // BCrypt with cost factor 12 return new BCryptPasswordEncoder(12); // OR: Argon2id (preferred for new systems) // return new Argon2PasswordEncoder(19456, 2, 1, 32, 4); // 19456KB memory, 2 iterations, 1 parallelism, 32-byte hash, 4 salt-length // OR: DelegatingPasswordEncoder (supports multiple, good for migration) // return PasswordEncoderFactories.createDelegatingPasswordEncoder(); // Encodes as {bcrypt}$2a$10$... - can verify any supported format } // Encoding and verifying String encoded = passwordEncoder.encode(rawPassword); // {bcrypt}$2a$12$... boolean matches = passwordEncoder.matches(rawPassword, encoded); // true // NEVER roll your own: // BAD: MessageDigest.getInstance("SHA-256").digest(password.getBytes()) // Missing: salt, iterations, work factor
```

Salting & Why It Matters

Without a salt, identical passwords produce identical hashes — enabling rainbow table attacks (precomputed hash-to-password mappings). BCrypt, Argon2id, and SCrypt automatically generate a cryptographically random salt per password and include it in the hash output. PBKDF2 requires you to generate and store the salt separately. The salt does not need to be secret — it just needs to be unique per password.

Symmetric Encryption: AES-GCM



AES-GCM = confidentiality (AES-CTR) + integrity (GHASH). AEAD — authenticated encryption with associated data.

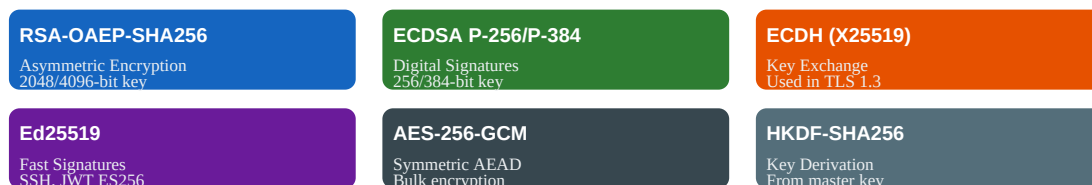
IV (nonce): 96-bit, unique per encryption. NEVER reuse IV with same key — catastrophic security failure.

Key: derive with HKDF from master key. Store master key in AWS KMS / HashiCorp Vault — never in source code.

AWS KMS GenerateDataKey: returns plaintext DEK for use + encrypted DEK for storage (envelope encryption).

```
// AES-256-GCM encryption in Java public class AesGcmEncryption { private static final int
KEY_SIZE = 256; private static final int IV_SIZE = 12; // 96 bits private static final int
TAG_SIZE = 128; // 128 bits public byte[] encrypt(byte[] plaintext, SecretKey key) throws
Exception { byte[] iv = new byte[IV_SIZE]; new SecureRandom().nextBytes(iv); // MUST be unique
per encryption! Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(TAG_SIZE, iv)); byte[] ciphertext =
cipher.doFinal(plaintext); // Prepend IV to ciphertext for storage byte[] output = new
byte[IV_SIZE + ciphertext.length]; System.arraycopy(iv, 0, output, 0, IV_SIZE);
System.arraycopy(ciphertext, 0, output, IV_SIZE, ciphertext.length); return output; } public
byte[] decrypt(byte[] encryptedData, SecretKey key) throws Exception { byte[] iv =
Arrays.copyOfRange(encryptedData, 0, IV_SIZE); byte[] ciphertext =
Arrays.copyOfRange(encryptedData, IV_SIZE, encryptedData.length); Cipher cipher =
Cipher.getInstance("AES/GCM/NoPadding"); cipher.init(Cipher.DECRYPT_MODE, key, new
GCMParameterSpec(TAG_SIZE, iv)); return cipher.doFinal(ciphertext); // Throws
AEADBadTagException if integrity check fails } }
```

Asymmetric Cryptography & Digital Signatures



Never use: RSA PKCS#1v1.5 (padding oracle), DSA (weak), DH<2048-bit, 3DES, RC4, MD5, SHA-1 for new systems.

```
// RSA key generation KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
kpg.initialize(2048); // 2048 minimum; 4096 for long-term keys KeyPair keyPair =
kpg.generateKeyPair(); // ECDSA signing (for JWTs, code signing) KeyPairGenerator ecKpg =
KeyPairGenerator.getInstance("EC"); ecKpg.initialize(new ECGenParameterSpec("secp256r1")); //
P-256 KeyPair ecKeyPair = ecKpg.generateKeyPair(); Signature signer =
Signature.getInstance("SHA256withECDSA"); signer.initSign(ecKeyPair.getPrivate());
signer.update(data); byte[] signature = signer.sign(); // Verify Signature verifier =
Signature.getInstance("SHA256withECDSA"); verifier.initVerify(ecKeyPair.getPublic());
verifier.update(data); boolean valid = verifier.verify(signature);
```

Key Management: AWS KMS & HashiCorp Vault

Feature	AWS KMS	HashiCorp Vault
Key Storage	HSM-backed, managed service	Shamir secret sharing, unseal keys
Encryption	GenerateDataKey + encrypt/decrypt	Transit secrets engine (AES-GCM)
Key Rotation	Automatic annual rotation	Manual or scheduled rotation
Access Control	IAM policies + key policies	Policies, AppRole, Kubernetes auth

Audit Logging	CloudTrail	Vault audit devices (file, syslog)
Dynamic Secrets	N/A (static keys)	Generate DB creds, AWS creds on-demand
Secret Storage	SSM Parameter Store / Secrets Mgr	KV secrets engine v1/v2
Envelope Encrypt.	Yes — native support	Via Transit + KV combination

```
// AWS KMS envelope encryption in Spring
@Service public class KmsEncryptionService {
    private final KmsClient kms;
    private final String keyId; // CMK ARN
    public EncryptedData encrypt(byte[] plaintext) {
        // Generate a data encryption key (DEK)
        GenerateDataKeyResponse dkResponse = kms.generateDataKey(
            GenerateDataKeyRequest.builder().keyId(keyId).keySpec(
                DataKeySpec.AES_256).build());
        SecretKey dek = new SecretKeySpec(dkResponse.plaintext().asByteArray(), "AES");
        byte[] encryptedDek = dkResponse.ciphertextBlob().asByteArray();
        // Use DEK to encrypt data locally (fast)
        byte[] ciphertext = aesGcmEncrypt(plaintext, dek);
        // Store: encryptedDek + ciphertext
        // Do NOT store plaintext DEK
        return new EncryptedData(encryptedDek, ciphertext);
    }
}
```

50+ Interview Questions & Answers

Q1. Why should you never store passwords in plaintext?

A single database breach exposes all users' passwords immediately. Users often reuse passwords — attacker gains access to their accounts on other sites. No legitimate use case requires storing plaintext passwords — hash comparison works for verification.

Q2. What is the difference between a hash function and encryption?

Hash: one-way function — cannot reverse to get original input. Deterministic for same input. Used for passwords, integrity checks. Encryption: two-way — can decrypt with the key. Used for data that needs to be recovered. Never use encryption for passwords.

Q3. Why are MD5 and SHA-1 unsuitable for passwords?

They are fast — GPUs can compute billions per second, enabling brute force. They are also cryptographically broken (collision attacks). Fast hashes have no work factor — cannot be tuned to keep pace with hardware improvements.

Q4. What is an adaptive hash function?

A hash function with a configurable work factor (cost/iterations) that can be increased as hardware improves. BCrypt: cost factor (2^{cost} iterations). Argon2id: time, memory, parallelism. Target: > 100ms to hash one password on your hardware.

Q5. What is BCrypt?

An adaptive password hashing algorithm based on the Blowfish cipher. Output format: \$2a\$12\$<22-char-salt><31-char-hash>. Always 60 characters. Cost factor (10-12 recommended). Auto-generates and embeds salt. Maximum input: 72 bytes.

Q6. What is Argon2id and why is it preferred?

Winner of the Password Hashing Competition (2015). Argon2id is the recommended variant (hybrid of Argon2i and Argon2d). Parameters: memory (19MiB), iterations (2), parallelism (1). Resistant to GPU and side-channel attacks. Spring Security 5.3+.

Q7. What is the BCrypt 72-byte limitation?

BCrypt only processes the first 72 bytes of the password. Passwords longer than 72 bytes are silently truncated. Mitigation: pre-hash with HMAC-SHA256 before BCrypt, or use Argon2id (no length limit). Most users don't hit this limit, but be aware.

Q8. What is a rainbow table attack?

Precomputed table of hash → password mappings. An attacker can look up a hash and find the corresponding password instantly. Defeated by salting — the salt makes each hash unique even for identical passwords, making precomputation infeasible.

Q9. What is salting?

Adding a random value (salt) to the password before hashing. Salt is unique per password (not per application). Stored alongside the hash. Makes rainbow tables infeasible. BCrypt/Argon2/SCrypt auto-generate and embed salts.

Q10. What is pepper?

A secret value added to all passwords before hashing, stored separately from the database (e.g., application config/secret manager). Unlike salt, pepper is the same for all passwords. Adds another layer: even with DB dump, attacker needs the pepper to crack.

Q11. What is DelegatingPasswordEncoder?

Spring's PasswordEncoder that prefixes the algorithm ID: {bcrypt}\$2a\$..., {argon2}\$argon2id\$.... Allows multiple algorithms simultaneously — encode with preferred, verify any supported. Enables transparent migration as better algorithms emerge.

Q12. How do you migrate passwords from MD5 to BCrypt?

At next login: verify MD5 hash, re-hash with BCrypt, update stored hash. Over time, all active users are migrated. For inactive users: force password reset on next login. DelegatingPasswordEncoder automates the re-encoding.

Q13. What is work factor tuning?

Adjust BCrypt cost or Argon2 parameters so hashing takes ~100ms on your production hardware. Too fast: vulnerable to brute force. Too slow: legitimate auth becomes slow. Benchmark regularly — increase work factor every few years as hardware improves.

Q14. What is AES-GCM?

AES in Galois/Counter Mode — an AEAD (Authenticated Encryption with Associated Data) cipher. Provides both confidentiality (AES-CTR encryption) and integrity (GHASH authentication tag). No need for separate MAC. Key sizes: 128 or 256 bits. IV: 96 bits.

Q15. What is AEAD?

Authenticated Encryption with Associated Data — encrypts and authenticates the ciphertext in one operation. Decryption fails if data was tampered. Examples: AES-GCM, ChaCha20-Poly1305. Preferred over encrypt-then-MAC (simpler, less error-prone).

Q16. Why should you never reuse an IV in AES-GCM?

Reusing IV + key in GCM allows an attacker to recover the GHASH key, break integrity protection, and potentially recover plaintext (XOR attack). Generate a fresh random 96-bit IV for every encryption. Use SecureRandom, never a counter.

Q17. What is envelope encryption?

Encrypting data with a Data Encryption Key (DEK), then encrypting the DEK with a Key Encryption Key (KEK). Store encrypted DEK alongside data. KEK lives in KMS. Allows key rotation (re-encrypt DEK with new KEK) without re-encrypting all data.

Q18. What is AWS KMS?

AWS Key Management Service — managed service for creating and controlling cryptographic keys. Keys stored in HSMS. Provides: GenerateDataKey, Encrypt, Decrypt, Sign, Verify. All operations are logged in CloudTrail. Keys never leave KMS in plaintext.

Q19. What is the difference between CMK and DEK in KMS?

CMK (Customer Master Key): the master key that never leaves KMS. Used to encrypt/decrypt DEKs. DEK (Data Encryption Key): generated by KMS, used locally to encrypt data. Short-lived, discarded after use. Only the encrypted DEK is stored.

Q20. What is HashiCorp Vault?

Open-source secrets management tool. Features: KV secrets storage, Transit secrets engine (encrypt/decrypt without exposing key), dynamic secrets (generate short-lived DB credentials), PKI (issue certificates), Kubernetes integration.

Q21. What is the Transit secrets engine in Vault?

Vault's encryption-as-a-service — your application sends plaintext, Vault returns ciphertext (or vice versa). The encryption key never leaves Vault. Enables encryption without managing keys in your application. Supports key rotation.

Q22. What is RSA-OAEP?

RSA with Optimal Asymmetric Encryption Padding — the correct RSA encryption scheme. Never use RSA PKCS#1v1.5 (padding oracle attack, used in DROWN attack). OAEP adds randomness and is IND-CCA2 secure. Key size: 2048-bit minimum, 4096 for long-term.

Q23. What is the difference between RSA encryption and RSA signature?

Encryption (RSA-OAEP): encrypt with public key, decrypt with private key — used to share secrets. Signature (RSA-PSS or PKCS#1v1.5): sign with private key, verify with public key — used to prove authenticity. Different operations, opposite key usage.

Q24. What is ECDSA?

Elliptic Curve Digital Signature Algorithm. Much smaller keys than RSA: 256-bit ECDSA $\approx$ 3072-bit RSA security. P-256 (secp256r1) and P-384 (secp384r1) are NIST-approved curves. Used in TLS certificates, JWT (ES256/ES384), code signing.

Q25. What is Ed25519?

An EdDSA signature scheme using Curve25519. Extremely fast, small keys (32 bytes), no random number dependency during signing (deterministic). Used in SSH, Signal protocol, JWT ES256. Preferred for new systems when NIST curves aren't required.

Q26. What is the difference between ECDH and ECDSA?

ECDH (Elliptic Curve Diffie-Hellman): key exchange — derive a shared secret without transmitting it. ECDSA: digital signatures — prove authenticity. TLS 1.3 uses ECDHE (ephemeral ECDH) for key exchange, ECDSA for certificate signatures.

Q27. What is key derivation?

Deriving one or more cryptographic keys from a master key or password. HKDF (RFC 5869): expand a single secret into multiple keys (Extract-then-Expand). PBKDF2: derive key from low-entropy password with many iterations. bcrypt/scrypt: for passwords.

Q28. What is HKDF?

HMAC-based Key Derivation Function (RFC 5869). Used to derive cryptographic keys from key material (not passwords). Two steps: Extract (make key material uniform) + Expand (derive multiple keys). Used in TLS

1.3, Signal protocol.

Q29. What is SecureRandom?

Java's cryptographically secure pseudo-random number generator (CSPRNG). Uses OS entropy sources (/dev/urandom on Linux). Use for: IV generation, salt generation, nonce generation, token generation. Never use Math.random() for security-critical values.

Q30. What is the difference between /dev/random and /dev/urandom?

On modern Linux: both use the same CSPRNG. /dev/random previously blocked when entropy pool was low (bad for production). /dev/urandom never blocks. SecureRandom uses /dev/urandom equivalent. Blocking behavior has been deprecated since Linux 5.6.

Q31. How do you securely generate tokens?

SecureRandom.nextBytes(32) — 256 bits of entropy. Encode as hex or Base64URL. This is sufficient for session tokens, API keys, CSRF tokens, password reset tokens. Never use UUID (only 122 random bits and some versions aren't random).

Q32. What is constant-time comparison and where is it needed?

Comparing secrets (HMAC signatures, tokens) with normal String.equals() leaks timing information — the comparison stops at the first mismatch. Use MessageDigest.isEqual() or constant-time comparison library. Needed for: HMAC verification, token validation.

Q33. What is timing attack?

An attack that infers secret information from the time taken to perform cryptographic operations. Variable-time operations (early-exit string comparison) reveal how many bytes match. Defense: constant-time algorithms for all secret comparisons.

Q34. What is encrypted at rest?

Data encrypted in storage (DB, file system, S3 bucket). Not just disk encryption — field-level encryption for sensitive data (SSN, credit card). Use AES-256-GCM. Key management via KMS. Required by PCI DSS, HIPAA for sensitive data.

Q35. What is the principle of key separation?

Different keys for different purposes: separate encryption key from signing key, separate keys per environment (dev/staging/prod), separate keys per data classification. Compromise of one key doesn't affect others.

Q36. What is key rotation?

Replacing an old key with a new one. For symmetric keys: decrypt with old, re-encrypt with new. For asymmetric: issue new key pair, update all systems, revoke old. AWS KMS supports automatic key rotation (annual). HashiCorp Vault supports manual rotation.

Q37. What is forward secrecy in the context of encryption?

Using ephemeral keys per session/operation so that compromise of the long-term key doesn't compromise past encrypted data. In TLS 1.3: ECDHE generates a new key per session. For stored data: periodically re-encrypt with new keys (key rotation).

Q38. What is the difference between encoding, encryption, and hashing?

Encoding (Base64): reversible transformation, no key — for data formatting, not security. Encryption: reversible with key — for confidentiality. Hashing: one-way — for integrity and password storage. Base64-encoding a password provides NO security.

Q39. What is PKCS#11?

A platform-independent API for cryptographic tokens (hardware security modules, smart cards). Java: JCE provider for PKCS#11 (SunPKCS11). Allows Java applications to use hardware-backed keys. AWS CloudHSM, YubiKey, and HSMs expose PKCS#11.

Q40. What is an HSM?

Hardware Security Module — a physical device providing tamper-resistant key storage and cryptographic operations. Keys never leave the HSM. Used for: CA root keys, payment systems, code signing, high-security applications. AWS KMS is backed by FIPS 140-2 Level 3 HSMs.

Q41. What is FIPS 140-2?

US government standard for cryptographic modules. Level 1-4 (increasing security). Level 2: tamper-evident. Level 3: tamper-resistant with key zeroization. Many enterprise customers require FIPS 140-2 validated modules. AWS KMS is Level 3. Java: use BouncyCastle FIPS provider.

Q42. What is the Java Cryptography Architecture (JCA)?

Java's framework for cryptographic operations. Providers implement algorithms: SunJCE (default), BouncyCastle (more algorithms, FIPS option), SunPKCS11 (HSM). Use Provider-independent APIs: Cipher, MessageDigest, KeyGenerator, Signature.

Q43. What is BouncyCastle?

A widely-used Java cryptographic library providing: additional algorithms beyond JCE, FIPS-certified module, TLS implementation (JSSE), X.509 certificate handling, PGP, CMS. Use for: Argon2, modern curves (X25519, Ed25519), JOSE operations.

Q44. What is Jasmyt?

Java Simplified Encryption library. Spring Boot integration: encrypt properties in application.yaml using ENC(encryptedValue). Key provided at startup via environment variable. Useful for encrypting DB passwords in config files.

Q45. What is key wrapping?

Encrypting a cryptographic key with another key (Key Encryption Key). AES Key Wrap (RFC 3394): wraps AES keys. Used in PKCS#12, JOSE (JWE key management). KMS GenerateDataKey: returns encrypted (wrapped) DEK for storage.

Q46. What is password-based encryption (PBE)?

Encrypting data using a key derived from a password. Use PBKDF2 to derive a key from password+salt, then use AES-GCM with derived key. Only as secure as the password strength. Consider using hardware keys (KMS) instead for server-side encryption.

Q47. What are Java KeyStore types?

JKS (Java KeyStore): Java proprietary, legacy. PKCS12 (.p12/.pfx): standard format (RFC 7292), Java 9+ default. JCEKS: supports more key types. BKS: BouncyCastle, used in Android. Use PKCS12 for new systems — compatible with openssl, keytool, and other tools.

Q48. What is secure memory handling for secrets in Java?

Use char[] instead of String for passwords (String is immutable, may stay in memory). Clear arrays after use: Arrays.fill(chars, '\0'). Use GuardedString in LDAP SDK. Avoid: logging secrets, passing as method parameters, storing in collections. Java doesn't guarantee GC timing.

Q49. What is Spring Security Crypto?

Spring's crypto module (spring-security-crypto): PasswordEncoder, TextEncryptor, BytesEncryptor, KeyGenerators. BytesEncryptor.standard() uses AES-256-CBC with PKCS5. BytesEncryptor.stronger() uses AES-256-GCM. TextEncryptor wraps BytesEncryptor for String encryption.